

Week 4: Progress Report

Technical progress:

- Locked in an end-to-end runnable notebook covering every Week 3 follow-up: per-class ROC-AUC, a 2023→2024 temporal split, the NaN-coordinate policy A/B, a meteorological replacement for CZ_TYPE, and a refined TabNet sweep with k-fold CV. All headline numbers are persisted to `reports/week4\_results.json` for citation without re-running the notebook.
- Added per-class one-vs-rest ROC-AUC for all four random-split models (default XGBoost, tuned XGBoost, TabNet baseline, tuned TabNet). Macro AUC: default XGB [TBD], tuned XGB [TBD], TabNet baseline [TBD], tuned TabNet [TBD].
- Implemented a temporal split (train 2023, test 2024). Re-fit the hierarchical lat/lon imputation on 2023 rows only so 2024 is fully held out. Tuned XGBoost moved from [TBD] (random split) to [TBD] (2023→2024); tuned TabNet moved from [TBD] to [TBD]. Delta confirms generalization across years.
- Settled the NaN-coordinate policy via apples-to-apples comparison: trained an "impute" variant and a "drop" variant of each tuned model, then evaluated both on the ****same**** native-coord test subset.
- Tuned XGBoost: impute [TBD] vs drop [TBD] (Δ [TBD]). Tuned TabNet: impute [TBD] vs drop [TBD] (Δ [TBD]). [Decision: TBD — keep impute / switch to drop].
- Engineered a meteorological replacement for CZ_TYPE: leave-one-out target-encoded class priors at two granularities — (STATE, MONTH) and WFO — Laplace-smoothed ($\alpha=10$) toward the global class prior. Tuned XGBoost without CZ_TYPE recovered [TBD]% of the ablation-induced macro-F1 drop using the priors; tuned TabNet recovered [TBD]%.
- Expanded TabNet tuning beyond the Week 3 coarse grid: held the coarse winner's `n\_d`/`n\_a`, `n\_steps`, and `lr` fixed and swept $\gamma \in \{1.0, 1.3, 1.5, 1.8\} \times \text{batch_size} \in \{512, 1024, 2048\}$ for 12 additional candidates. Best refined config: γ = [TBD], batch_size= [TBD].
- Ran 3-fold stratified CV on the top-3 refined finalists for a tighter macro-F1 estimate (9 additional TabNet fits). CV winner retrained on full training set and evaluated on held-out test set: macro F1 = [TBD] (Δ [TBD] vs coarse-tuned TabNet, Δ [TBD] vs tuned XGBoost).
- Persisted every metric (random-split, ablation, sweep best, ROC-AUC, NaN policy, CZ_TYPE replacement, refined tuning, temporal split) into `reports/week4\_results.json` so downstream reporting does not require re-running the notebook.

Obstacles:

- Cannot execute the full notebook locally without GPU. The refined TabNet sweep adds 12 fits, the k-fold CV adds 9 fits, the temporal-split TabNet adds 1 fit, the NaN-policy variant adds 1 fit, and the CZ_TYPE-replacement TabNet adds 1 fit — roughly 24 additional TabNet fits beyond Week 3's already CUDA-only sweep. Final numbers will be filled in once the notebook is run on a CUDA-enabled machine.
- The CZ_TYPE replacement adds 20 features (10 priors per grouping) on top of the 7 surviving features, raising the dimensionality from 8 to 27. LOO encoding plus $\alpha=10$ smoothing should suppress self-leakage, but a high feature count combined with target-derived columns is still a known overfitting risk. Will need to compare train vs test F1 carefully when numbers are in.

- The refined TabNet sweep assumes the coarse winner's region is globally promising. If the coarse sweep landed in a local optimum, freezing `n\_d`/`n\_a`, `n\_steps`, `lr` will not escape it. A larger joint sweep would be safer but was not run for runtime reasons.
- The temporal split has unequal class counts across 2023 and 2024 (no stratification — the year is the split variable). Some rare classes may be under-represented in either side, which can inflate per-class variance in the macro-F1 comparison versus the random split.
- NaN-coordinate policy comparison is run on a smaller (~60%) test subset to keep the head-to-head fair. Macro F1 numbers in this section are not directly comparable to the headline (full-test) numbers — only to each other.

Plan for next week:

- Run the locked-in notebook end-to-end on CUDA, fill in every `[TBD]` in this report, and finalize the headline number table for the Phase 2 paper.
- Choose the final production configuration based on the locked-in numbers: (a) impute vs drop for coordinates, (b) with-CZ_TYPE vs CZ_TYPE-replacement features, (c) tuned XGBoost vs refined TabNet.
- Begin the Phase 2 final write-up: pull the confusion matrices, ROC curves, and feature-importance figures directly from the notebook into the paper.
- If the refined-TabNet test F1 still trails tuned XGBoost, document the negative result honestly and discuss why attention-based feature selection did not pay off on this dataset (likely: low feature count, strong tabular signal already captured by tree splits).
- Investigate whether combining the CZ_TYPE replacement priors with `CZ\_TYPE\_ENC` (rather than as a strict replacement) yields a further improvement, since the priors and CZ_TYPE encode different information.

Week 3: Progress Report

Technical progress:

- Reused the Week 2 tuned XGBoost configuration (learning_rate=0.1, max_depth=9, n_estimators=500, subsample=0.7) instead of re-running the 192-candidate grid search. Commented out the search cell so it stays in the notebook for reproducibility without adding ~16 minutes per execution.
- Ran the TabNet baseline on a CUDA-enabled machine and compared macro F1 against both the default and tuned XGBoost models using the same train/test split and feature set.
- Implemented a CZ_TYPE ablation study: retrained default XGBoost, tuned XGBoost (with Week 2 best params), and TabNet on an eight-feature set with CZ_TYPE_ENC removed to measure its true contribution.

- Reported per-class classification reports and a side-by-side delta table to see whether each model can hold macro F1 on purely meteorological features (MONTH, HOUR, STATE_ENC, BEGIN_LAT, BEGIN_LON, DURATION_MIN, WFO_ENC).
- Computed a new feature importance ranking for the tuned XGBoost without CZ_TYPE to see which features the model leans on once the dominant non-meteorological signal is removed.
- Began TabNet hyperparameter tuning: grid over n_d=n_a in {8, 16, 32}, n_steps in {3, 5, 7}, and learning rate in {1e-2, 2e-2, 5e-2}, for 27 candidate configurations.
- Used a stratified 80/20 carve-out of the training set for tuning validation so the held-out test set stays untouched during model selection.

Obstacles:

- Re-executing the whole notebook end-to-end is slow. The TabNet tuning cell alone is ~30 full TabNet fits on ~94k rows (27 sweep configs plus one final retrain), and nbconvert does not stream cell output, so intermediate progress is not visible until completion.
- Still need to decide whether to impute missing BEGIN_LAT/BEGIN_LON via the hierarchical STATE/CZ_FIPS to STATE/WFO to STATE median fill already wired up, or drop NaN-coordinate rows before TabNet training. The current path imputes, which keeps rows but may bias coordinate-sensitive models.
- TabNet sweep uses a single validation split rather than k-fold CV to keep runtime tractable. Results may have higher variance than the XGBoost grid search numbers and are not strictly comparable.
- The `model = grid_search.best_estimator_` line in the original tuning cell was commented out, which silently broke downstream references. Had to explicitly rebuild and fit the tuned model from `best_params` so later cells do not rely on the commented-out grid search.

Plan for next week:

- Complete a full end-to-end execution of the updated notebook and lock in concrete macro F1 numbers for the ablation study and TabNet tuning.
- If the ablation shows a large drop without CZ_TYPE, investigate feature engineering that could replace it with meteorological signal (e.g., state-by-month priors, WFO-level climatology) rather than relying on a non-meteorological column.
- Expand TabNet tuning with gamma and batch_size once the coarse sweep identifies a promising region, and consider k-fold CV on the finalist configs.
- Settle the NaN-coordinate policy (impute vs. drop) by comparing both on the tuned XGBoost and tuned TabNet, and document the decision.

Week 2: Progress Report

Technical progress:

- Tuned XGBoost hyperparameters via grid search with 5-fold stratified cross-validation over 192 candidate configurations (learning rate, max depth, n_estimators, subsample)
- Best configuration: learning_rate=0.1, max_depth=9, n_estimators=500, subsample=0.7

- Tuned XGBoost achieved macro F1 of 0.90 on held-out test set, up from 0.89 default (+0.0116 improvement)
- Hail vs Thunderstorm Wind confusion reduced from 2,358 to 1,865 misclassifications with tuning (Hail as TStorm Wind: 1,431 to 1,081; TStorm Wind as Hail: 927 to 784)
- Feature importance shifted heavily toward CZ_TYPE under tuning (0.65 to 0.90), confirming the concern raised in Week 1 about over-reliance on this feature
- Implemented baseline TabNet model using pytorch-tabnet with GPU acceleration (CUDA)
- TabNet configured with n_d=8, n_a=8, n_steps=3, batch_size=1024, early stopping with patience=15
- Passed categorical feature indices (STATE_ENC, CZ_TYPE_ENC, WFO_ENC) to TabNet for learned embeddings rather than treating them as continuous

Obstacles:

- XGBoost tuning grid search (960 total fits) caused memory warnings from joblib worker timeouts, though it completed successfully
- CZ_TYPE importance increased from 0.65 to 0.90 after tuning, meaning the tuned model leans even more heavily on a non-meteorological feature. This needs to be addressed before final evaluation
- TabNet results are pending execution of the notebook on a CUDA-enabled machine
- Unsure if we should estimate the coordinates to replace NaN entries in the data set or drop the entries containing NaN values for the coordinates column since TabNet cannot handle NaN values.

Plan for next week:

- Run and evaluate TabNet baseline, compare macro F1 against both XGBoost models
- Ablation study: retrain models with CZ_TYPE removed to measure its true contribution and determine if the model can maintain performance on purely meteorological features
- Begin TabNet hyperparameter tuning (n_d/n_a width, n_steps, learning rat

Week 1: Progress Report

Technical progress:

- Engineered seven contextual features: month, hour of day, state, latitude, longitude, duration in minutes, CZ_TYPE, and WFO
- Parsed BEGIN_DATE_TIME and END_DATE_TIME timestamps to compute event duration, verified no negative or absurd values
- Applied label encoding to categorical features (STATE, WFO, CZ_TYPE) for XGBoost consumption
- Verified feature distributions: confirmed duration separates classes cleanly across five tiers (Drought median 30 days down to Hail/Thunderstorm Wind median 0 minutes)
- Performed stratified 80/20 train/test split preserving class proportions
- Trained baseline XGBoost classifier with default hyperparameters and tree_method='hist' to handle missing lat/long values natively
- Achieved macro F1 score of 0.89 on held-out test set

- Generated per-class classification report and confusion matrix to identify error patterns
- Identified Hail vs Thunderstorm Wind as the dominant confusion (2,337 misclassifications, roughly 70% of all errors), confirming the phase 2 test case predicted in the proposal
- Analyzed feature importance: CZ_TYPE (0.65), Duration (0.11), and geographic features (lat/long, 0.13 combined) drive predictions

Obstacles:

- Spent significant time refactoring feature extraction code because our initial implementation did not use pandas. After using pandas we were able to simplify the code significantly.
- CZ_TYPE may be overly important. CZ_TYPE is not a purely meteorological feature.
- So far it looks like XGBoost is actually performing a lot better than anticipated in the proposal (this is fine, just worth noting).

Plan for next week:

- Investigate the impact of the CZ_TYPE on F1 score
- Tune XGBoost model as described in the proposal (learning rate, max depth, number of estimators, and subsample ratio) to achieve a baseline to compare to TabNet
- Train and evaluate default TabNet model